

题目链接

本题是2009年ICPC欧洲区域赛中欧赛区的A题

题意

你有一个会计帮忙记账，他将你的原始账单写入汇总文件中。每一个原始账单包含若干item， item → name = number或者item → name = item + item。会计对单个账单入账时可能将此账单所有相同的name改写，但是不同的name不会被改写成相同的，并且会计不会修改number。会计有可能遗漏部分账单，导致其信息没被追加到汇总文件中。已知所有原始账单信息（账单数k≤50000）以及汇总账单信息，请判定每个账单是否遗漏，一定遗漏输出NONE，可能没遗漏则输出其在汇总文件中首次匹配到的行号。所有账单的总结点数以及汇总文件的结点数都不超过3,000,000。

分析

很好的一道AC自动机的综合应用题，需要对AC自动机有充分的理解，并且细节很多。

官方题解：

The problem statement can be easily reformulated as the following pattern matching problem: you are given an alphabet consisting of two separate parts: terminals, denoted by small letters, and nonterminals denoted by large letters. We consider two strings over this alphabet similar iff there exists a permutation of all the nonterminals which transforms one string into another. For example string "aXbYcX" is similar to "aZbVcZ", but it is not similar to "aZbZcZ". We say that one string occurs in another one if it is similar to its substring. The task is to find (first) occurrences of a given set of strings in a large text. The defined above similarity is an equivalence relation thus it might be useful to assign a representative (canonical) string to each equivalence class. An obvious choice for such representation would be the the (lexicographically) smallest string but it turns out to be not very convenient to work with. Therefore, the representation that we are going to use will actually be defined over a completely different alphabet (so in fact it is not going to belong to the given equivalence class). If you think about it for a moment, the only important thing about the nonterminals is equality, we need to know which of them are the same and which are not. Therefore, we replace each nonterminal with a number, which is the distance from the previous occurrence of the same nonterminal. In case the nonterminal is completely new, we use INF (or, more precisely, a number which is greater or equal to the length of prefix read so far). So, we will encode "aXbYcX" (and "aZbVcZ" as well) as "a2b4c4". How do we decide if one string is similar to another? In the ideal world (that is, if we had chosen representative to actually be a representative) it would suffice to compare representatives. This works quite well for two strings of same length, however we need to find occurrence of one string in another one, which is possibly longer, and thus the fragment may contain numbers larger than the length of the prefix. For example a text "YcaXbYcX" is encoded as "1ca4b6c4", which makes it difficult to realize that it contains "a2b4c4". To check if we have found an occurrence, you just need to replace all numbers greater than their position by their position itself, which will effectively convert "a4b6c4" to "a2b4c4", and now you can compare it byte by byte. This leads to the following algorithm (I hope that reader is familiar with the Aho-Corasick Algorithm, otherwise it is not going to make much sense):

- Convert patterns and text from the input to the language of terminals and nonterminals. This can be done using hashtables in roughly linear time. Care must be taken to separate lines from each other.
- Replace each nonterminal with the distance from its previous occurrence, or its position, if there is no previous occurrence. Using a hash table, you can do this in linear time as well.
- Put all patterns in a TRIE, just like in the Aho-Corasick algorithm. If you store children in hashtables you can do that in linear time.
- Rules for navigating in this tree are almost the same as in the original algorithm, unless you try to visit child N, where N is greater or equal to the depth of current node. Then you should try to visit child whose number is the same as the depth of current node.
- For each node of the TRIE find the backlink edge using a single BFS pass over the TRIE.
- For each node corresponding to one of the pattern we are looking for, add its number to the list of matches.
- Once you have built the automaton, you just perform usual Aho-Corasick Algorithm, remembering that there is a special rule for numbers greater than current depth. Whenever you visit a node that has not null pointer to the list of matches, and is not marked as reported you report all of them recursively, and mark all of them as reported. This is necessary to avoid n^2 complexity.

We assumed the program may use a STL map which increases the complexity from O(n) to O(nlgn).

参照官方题解可以写出代码，这里说一些细节：

《训练指南》给出的AC自动机模板是将结点0作为根结点，本题用结点1作为根结点（结点0代表空指针）合适一些；

建立trie时还需要辅助数组记录结点深度信息，d[u]表示的是结点u的子节点深度，因此根结点深度为d[1]=1，这样做在bfs计算失败指针和最后遍历答案时将带来便利；

为区分终结符（自然数）和非终结符（name）并在结构上统一两者，可以将终结符s的key设定为其自然数的相反数（即-atoi(s)），非终结符的key设置为正值（按官方题解定义）；

对每个item，除了自身具有两/三个结点外，还需要两个结点，EQ和ED（EQ代表等号，ED代表换行），用这两个分隔符实现结点结构及item间的区分，两者结点定义为很大的负数即可；

由于每个item多加了两个给点，实际结点数可能翻倍，题目交代总结点数不超过3,000,000，代码里需要定义的常量是6,000,000；

AC代码

```
#include <iostream>
#include <string>
#include <unordered_map>
#include <vector>
using namespace std;

#define EQ -100000001
#define ED -100000002
#define N 50010
#define L 600060
unordered_map<int, int> ch[L]; int d[L], f[L], vis[L], q[L], ans[N], m[N], n, t; vector<int> g[L]; string s;

void solve() {
    cin >> n;
    f[t = 1] = vis[1] = 0; ch[d[1] = 1].clear();
    for (int i=1; i<=n; ++i) {
        unordered_map<string, int> p; int a = ans[i] = 0, x = 1; char _; cin >> m[i];
        for (int j=0; j<m[i]; ++j) {
            cin >> s; isdigit(s[0]) ? q[a++] = -atoi(s.c_str()) : (q[a] = p.count(s) ? a-p[s] : a+1, p[s] = a++);
            q[a++] = EQ;
            cin >> _ >> s; isdigit(s[0]) ? q[a++] = -atoi(s.c_str()) : (q[a] = p.count(s) ? a-p[s] : a+1, p[s] = a++);
            if (cin.peek() == '\n') { q[a++] = ED; continue; }
            cin >> _ >> s; isdigit(s[0]) ? q[a++] = -atoi(s.c_str()) : (q[a] = p.count(s) ? a-p[s] : a+1, p[s] = a++);
            q[a++] = ED;
        }
        for (int j=0, p=2; j<a; x = ch[x][q[j++]], ++p)
            if (!ch[x].count(q[j])) vis[ch[x][q[j]] = ++t] = 0, d[t] = p, ch[t].clear(), g[t].clear();
        g[x].push_back(i);
    }
    int head = 0, tail = 0;
    for (unordered_map<int, int>::iterator it = ch[1].begin(); it != ch[1].end(); ++it) f[q[tail++] = it->second] = 1;
    while (head < tail) {
        int u = q[head++];
        for (unordered_map<int, int>::iterator it = ch[u].begin(); it != ch[u].end(); ++it) {
            int v = f[u]; while (v && !ch[v].count(min(d[v], it->first))) v = f[v];
            f[q[tail++] = it->second] = v ? ch[v][min(d[v], it->first)] : 1;
        }
    }
    unordered_map<string, int> p; int a = 0; char _; cin >> m[0];
    for (int i=0; i<m[0]; ++i) {
        cin >> s; isdigit(s[0]) ? q[a++] = -atoi(s.c_str()) : (q[a] = p.count(s) ? a-p[s] : a+1, p[s] = a++);
        q[a++] = EQ;
        cin >> _ >> s; isdigit(s[0]) ? q[a++] = -atoi(s.c_str()) : (q[a] = p.count(s) ? a-p[s] : a+1, p[s] = a++);
        if (cin.peek() == '\n') { q[a++] = ED; continue; }
        cin >> _ >> s; isdigit(s[0]) ? q[a++] = -atoi(s.c_str()) : (q[a] = p.count(s) ? a-p[s] : a+1, p[s] = a++);
        q[a++] = ED;
    }
    for (int i=0, x=1, p=2; i<a; ++i) {
        while (x && !ch[x].count(min(d[x], q[i]))) x = f[x];
        x = x ? ch[x][min(d[x], q[i])] : 1;
        for (int j=x; j && !vis[j]; vis[j] = 1, j = f[j])
            for (int k=g[j].size()-1; k>=0; --k) if (!ans[g[j][k]]) ans[g[j][k]] = p;
        if (q[i] == ED) ++p;
    }
    for (int i=1; i<=n; ++i) ans[i] ? cout << ans[i]-m[i] << endl : cout << "NONE" << endl;
}

int main() {
    ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
    int t; cin >> t;
    while (t--) solve();
    return 0;
}
```